
Software Engineering Project Report

Supermarket System

Carried out by group:

Amanda PAULA FONTENELE

João Vitor DE OLIVEIRA FRAGA

June 4, 2026

Contents

1	Design Decisions	1
1.1	Package structure	1
1.2	Requirement impact on design	1
2	Design Patterns	3
2.1	Strategy Pattern — Customer Discount Plans	3
2.2	Strategy Pattern — Category Pricing Policies	3
2.3	Strategy Pattern — Delivery Fee Policy	4
2.4	Factory Pattern — PlanFactory	4
2.5	Observer Pattern — Low Stock Notification	5
3	Design Analysis	6
3.1	Advantages	6
3.2	Limitations	6
4	UML Diagrams	7
4.1	Class Diagram	7
4.2	Use Case Diagram	7
4.3	Successful Checkout Sequence Diagram	7
4.4	Delivery Request Sequence Diagram	7
4.5	Payment Failure Sequence Diagram	12
5	Testing and Test Scenario	13
5.1	Test scenario description	13
5.2	How to run the CLUI scenario	13
5.3	JUnit tests	14
6	Workload Distribution	16

Chapter 1

Design Decisions

1.1 Package structure

Our Supermarket Checkout System is divided into nine main packages, one test package, one test scenario file, and one configuration file named `my_supermarket.ini`. The packages were defined according to the system specifications, grouping classes with similar responsibilities and related requirements in the same package. This organization makes the project easier to write, understand, test, and improve.

The model package was the first package to be implemented. It contains the simplest entities of the system and represents the main objects used by the supermarket checkout system. This package includes the logic for users and items in the system. The `Customer`, `Cashier`, and `Manager` classes extend the abstract `User` class that has a common structure for all users. `Item` and `Category` classes represent the products sold by the supermarket and their categories.

The pricing package contains the discount logic of the system, this package was designed using the Strategy Pattern. It includes the `CostumerPlan` interface and its concrete implementations: `NormalPlan`, `PrimePlan`, and `PlatinumPlan`. The package also contains the category discount policies, represented by `CategoryDiscountPolicy` interface, and `NoCategoryDiscount` and `PercentageCategoryDiscount` implementations.

The factory package contains the `PlanFactory` class. This class is responsible for creating the correct customer discount plan based on a string received from the command line. For example, when the user types `subscribeToPlan prime`, the factory creates a `PrimePlan` object.

The checkout package contains the classes responsible for the shopping cart and bill computation. It includes `Cart`, `CartLine`, `Bill`, and `CheckoutSession`. This package directly supports the cash register scenario described in the project document.

The delivery package contains the logic related to home delivery. It includes `DeliveryRequest`, `DeliveryPolicy`, and `StandardDeliveryPolicy`. The `DeliveryRequest` stores the delivery address and the estimated distance. The `DeliveryPolicy` interface and `StandardDeliveryPolicy` implementation define how delivery fees are computed.

The payment package contains the payment system. It includes `BankCard`, `POS`, `TAS`, `PaymentResult`, and `SimulatedPaymentOutcome`. The `POS` represents the payment terminal used at the checkout, it receives the card number, PIN, and amount, then contacts the `TAS`, which represents the bank authorization system.

The observer package implements the Observer Pattern for low-stock notifications. It contains `StockObserver`, the observer `ManagerNotifier`, and `LowStockNotifier`. If a perishable item, such as meat or dairy, falls below its threshold, the `LowStockNotifier` notifies the registered observers.

The system package contains the `SupermarketSystem` class, which coordinates the whole application. This class connects all other packages and implements the main operations of the system, such as login, setup, registering users, adding items, starting checkout, scanning items, computing bills, processing payments, requesting delivery, showing inventory, and showing revenue.

The cli package contains the `SupermarketCLI` class. This class implements the command-line interface required by the project. The test package contains the JUnit tests for the most important methods of each class. The file `testScenario1.txt` contains a sequence of CLUI commands used to reproduce an integration test scenario. Finally, the file `my_supermarket.ini` is the initial configuration file. It is used to load the default supermarket configuration.

1.2 Requirement impact on design

The requirements had a direct impact on the architecture of our Supermarket Checkout System. We divided the system into packages according to the main responsibilities described in the project specification.

The model package was influenced mainly by the requirements related to customers, users, items, and categories. The user classes allow the system to distinguish between the different possible roles. The Category class allows the system to distinguish between the different possible categories of items.

The pricing package was created because several requirements concern discounts. The system must support customer discount plans and pricing policies, and the owner must be able to introduce or modify them. These requirements influenced the use of the Strategy Pattern, since different plans and category policies can be represented as interchangeable strategies.

The factory package was introduced because customer discount plans are created from command-line input. This motivated the use of a PlanFactory, which centralizes plan creation and avoids spreading object creation logic across the system.

The checkout package was designed around the cash register scenario. The document describes that the cashier records items and quantities, the checkout displays item prices, and then the system computes the total bill. Therefore, this separation allows the checkout process to focus only on scanning items and computing the bill.

The delivery package was created because the requirements state that customers may request home delivery, and that delivery fees must depend on total weight and distance. In our implementation, the address argument is interpreted as a delivery zone (near, medium, or far). This decision still allows the system to compute delivery fees based on distance while keeping the implementation simple and reproducible.

The payment package was influenced by the requirements describing the POS and the bank transaction authorization system. The POS represents the terminal used by the cashier, while the TAS is responsible for checking the bank card, verifying available balance, and authorizing or refusing the transaction. This separation follows the project description, which presents the POS and TAS as different components.

The observer package was influenced by the inventory management requirement. The system must maintain a live stock count, decrement the inventory after a sale and notify a manager or supplier module when perishable items fall below a configurable threshold. This led to the use of the Observer Pattern.

The system package contains the SupermarketSystem class, which coordinates the whole application. This design decision was influenced by the need to connect many different requirements. Instead of allowing the CLI to directly manipulate all objects, the CLI calls methods in SupermarketSystem, and this class enforces the system rules.

The requirements also influenced our decisions about permissions, since the command list specifies that some commands are reserved for specific users. The login system was designed with a single current user, so the system only stores one currentUser at a time. This avoids the complexity of managing multiple simultaneous sessions. To change roles, the user can log out and then log in with another account.

Another design decision concerns the setup command. We decided that it should be executed only at the beginning of the system execution, since allowing setup at any time could reset users, inventory, revenue, and an active checkout session, which would be unsafe and inconsistent with normal supermarket operation.

Chapter 2

Design Patterns

The system uses three main design patterns: Strategy, Factory, and Observer. The patterns helped us avoid large conditional structures inside classes and kept each responsibility separated in its own package.

2.1 Strategy Pattern — Customer Discount Plans

The first use of the Strategy Pattern is in the customer discount plans. The project requires the system to support at least three plans: normal, prime, and platinum. Each plan has a different pricing rule: the normal plan has no discount, the prime plan gives a 20% discount only when the item total is greater than or equal to 50 euros, and the platinum plan gives a 30% discount regardless of the total. Prime and platinum also affect delivery fees differently.

To avoid putting all these rules inside CheckoutSession or SupermarketSystem with several if/else statements, we created the CostumerPlan interface:

```
package pricing;

public interface CostumerPlan {
    String getName();

    double getAnnualFee();

    double costumerDiscount(double total);

    double deliveryDiscount(double deliveryFee);
}
```

Figure 2.1: CostumerPlan interface

The concrete strategies are: NormalPlan, PrimePlan and PlatinumPlan. Each class implements its own discount behavior. This design solves the problem required by R5 and R5b: the system supports multiple discount plans and can be extended with new plans. If a new plan is added later, such as StudentPlan or GoldPlan, we only need to create a new class implementing CostumerPlan.

2.2 Strategy Pattern — Category Pricing Policies

The second use of the Strategy Pattern is in the category-level pricing policies. The project requires the supermarket to apply specific pricing policies to item categories, such as a 10% reduction on fruit and vegetables or a 5% reduction on dairy products.

To represent this, we created the CategoryDiscountPolicy interface:

```
package pricing;

public interface CategoryDiscountPolicy {
    double categoryDiscount(double total);
}
```

Figure 2.2: CategoryDiscountPolicy interface

The concrete strategies are NoCategoryDiscount and PercentageCategoryDiscount. Each Category has its own CategoryDiscountPolicy. By default, a category starts with NoCategoryDiscount. When the manager uses the command setCategoryDiscount <discountPercent> the system replaces the current policy of the category with a PercentageCategoryDiscount(discountPercent).

This design avoids coding category discounts inside other classes. This satisfies R6 and R6b because it allows the owner to introduce new types of pricing policies for newly introduced categories of products, or to modify existing ones.

2.3 Strategy Pattern — Delivery Fee Policy

The third use of the Strategy Pattern is in the delivery fee calculation. The project requires delivery fees to depend on the total weight of the purchase, the delivery distance, and the customer plan. To isolate this rule, we created the DeliveryPolicy interface:

```
package delivery;

import pricing.CustomerPlan;

public interface DeliveryPolicy {
    double computeFee(double weight, double distanceKm, CustomerPlan plan, double itemsTotal);
}
```

Figure 2.3: DeliveryPolicy interface

The StandardDeliveryPolicy class implements the default delivery rule. If the delivery rule changes in the future, for example to include peak-hour fees, eco-friendly slots, or different delivery zones, the owner can add another implementation of DeliveryPolicy without modifying the system logic.

2.4 Factory Pattern — PlanFactory

The Factory Pattern is used in the creation of customer discount plans. The interface receives plan names as strings and needs to create an actual object. To avoid creating plan objects directly in SupermarketSystem with several if/else blocks, we created the PlanFactory class:

```
package factory;

import pricing.CustomerPlan;

public class PlanFactory {
    public CustomerPlan createPlan(String plan) {
        if (plan == null) {
            throw new IllegalArgumentException("Plan null");
        }

        String planName = plan.toLowerCase();

        switch (planName) {
            case "normal":
                return new NormalPlan();

            case "prime":
                return new PrimePlan();

            case "platinum":
                return new PlatinumPlan();

            default:
                throw new IllegalArgumentException("Plan: " + planName + " does not exist");
        }
    }
}
```

Figure 2.4: PlanFactory class

The factory centralizes the creation of plans. This is useful because SupermarketSystem does not need to know the construction details of each plan. This design also supports extensibility. If a new plan is introduced, we add a new class implementing DiscountPlan and update PlanFactory to recognize the new plan name.

2.5 Observer Pattern — Low Stock Notification

The Observer Pattern is used for low-stock notifications. The project requires the system to maintain live stock counts and notify a manager or supplier module when perishable items, such as meat or dairy, fall below a configurable threshold.

StockObserver is the observer interface. ManagerNotifier is a concrete observer, it receives the notification and displays a warning message. LowStockNotifier is responsible for storing the list of observers and notifying them. The notification is triggered only after a successful payment, because the inventory should only be decremented after a finalized sale.

This design avoids coupling the stock update logic to a specific notification mechanism. In the current implementation, the system notifies a manager, but new observers could be added later without changing the stock update or checkout logic.

Chapter 3

Design Analysis

3.1 Advantages

The main advantage of the solution we built is the separation of responsibilities among the different packages. The command-line interface is separate from the system logic, and each package has a specific function. For example, we have the checkout package, which is responsible for shopping carts, cart items, invoices, and checkout sessions, while the payments package handles credit card payments through the POS and TAS components. This makes it easier and more intuitive to understand, test, and modify the code.

Another advantage is the extensibility of the pricing logic, since customer discount plans are implemented through the Strategy Pattern, so that the checkout process does not depend on specific plan classes, such as PrimePlan or PlatinumPlan. If it is necessary to add a new customer plan, it can be implemented as a new class that implements the DiscountPlan interface. The same concept is used for category-level pricing policies, allowing each category to have its own discount policy.

The payment module is also structured to reflect the project's architecture. The POS and the TransactionAuthorizationSystem are separate. The POS represents the terminal used during checkout, while the TAS represents the bank-side authorization system. This separation makes the payment flow clearer and more closely aligned with the actual project description.

Inventory management is also improved through the use of the Observer Pattern. The system does not rely directly on a specific notification class; instead, when a perishable item falls below its threshold following a successful sale, the LowStockNotifier notifies the registered observers. This allows for the addition of other notification modules in the future, such as a supplier notifier, without altering the checkout logic.

In conclusion, the CLUI and the runTest command facilitate manual system testing, allowing you to write complete test scenarios in text files and run them from the command line. This capability is useful for verifying normal checkout flows, delivery requests, payment failures, and low-stock notifications.

3.2 Limitations

The solution also has some limitations. First, the system supports only one active checkout session at a time; while this is acceptable for a command-line project, it is not sufficient for a real supermarket with multiple checkouts operating in parallel.

Second, data is stored only in memory, so users, items, inventory levels, customer plans, and revenue are lost when the program is closed. A real-world application would require persistence, for example, through files or a database.

Third, the delivery address is simplified. Instead of using a real geographic address and calculating the actual distance, the implementation uses predefined delivery zones: near, medium, and far. This makes the system deterministic and easy to test; if we were to use geographic addresses, we would have to use a more complex system than the one we've already implemented, so ours ends up being less realistic than a real delivery system.

Another limitation is that the annual fee for customer plans is added directly to the supermarket's revenue when a customer signs up for a plan. A more comprehensive implementation could process this fee at the POS as a separate payment transaction. The requirement for advanced dynamic logistics has not been implemented; currently, the delivery module calculates a delivery fee based on weight, distance, and the customer's plan, but it does not manage delivery schedules, vehicle capacity, peak-hour pricing, or eco-friendly delivery times.

Finally, the system focuses on the mandatory CLUI commands and the key requirements of object-oriented design. Some behaviors have been simplified to ensure that the implementation is clear and easy to maintain within the scope of the project.

Chapter 4

UML Diagrams

This section presents the UML diagrams used to describe the structure and behavior of the Supermarket Checkout System. The diagrams include a class diagram, a use case diagram, and sequence diagrams for the main execution scenarios. All UML diagrams were created using PlantUML.

4.1 Class Diagram

The class diagram illustrates the main structure of the implementation; it shows the packages used in the project and the most important classes in each package. The `model` package contains the domain entities, such as `User`, `Manager`, `Cashier`, `Customer`, `Category`, and `Item`. The `checkout` package, on the other hand, contains the classes responsible for the checkout process, including `Cart`, `CartLine`, `Bill`, and `CheckoutSession`.

The diagram also shows the main design patterns used in the project. The `DiscountPlan`, `CategoryDiscountPolicy`, and `DeliveryPolicy` interfaces represent implementations of the Strategy Pattern. The `PlanFactory` class represents the Factory Pattern. The `StockObserver`, `ManagerNotifier`, and `LowStockNotifier` classes represent the Observer Pattern. Finally, the payments package illustrates the separation between the POS and the `TransactionAuthorizationSystem`, following the payment scenario described in the requirements.

4.2 Use Case Diagram

The use case diagram shows the main actors that interact with the system via the command-line interface. The main actors are `Manager`, `Cashier`, `Customer`, and `Generic User`.

The `Manager` can do the following: configure the system, register cashiers and customers, add items, restock items, set discounts by category, view inventory, and view revenue. The `Customer`, on the other hand, can sign up for a discount plan and request delivery for their next purchase. The `Cashier` can start a checkout process, scan items, calculate the total, simulate payment results, and initiate payment. The `Generic User` can use commands that do not require authentication, such as `help` and `runTest`.

Viewing this diagram makes it easier to better understand the permissions associated with each CLUI command

4.3 Successful Checkout Sequence Diagram

The successful checkout sequence diagram describes the main flow of a successful sale. It shows the cashier logging in, initiating a checkout for a registered customer, scanning the items, and asking the system to calculate the total. After that, the `CheckoutSession` uses the `Cart` to calculate the item totals and applies the active pricing policies and the customer's discount plan.

After calculating the total, the cashier initiates payment. The `SupermarketSystem` sends the payment request to the POS, which verifies the card and PIN and contacts the `TransactionAuthorizationSystem`. If the TAS authorizes the transaction, the payment is successfully completed. Finally, the system updates the revenue, reduces the inventory of the purchased items, triggers a low-stock notification if necessary, prints the receipt, and ends the checkout session.

4.4 Delivery Request Sequence Diagram

Moving on to the sequence diagram for the delivery request, it shows how a customer submits a pending delivery request. The customer logs in and executes the `requestDelivery` command; the system then converts

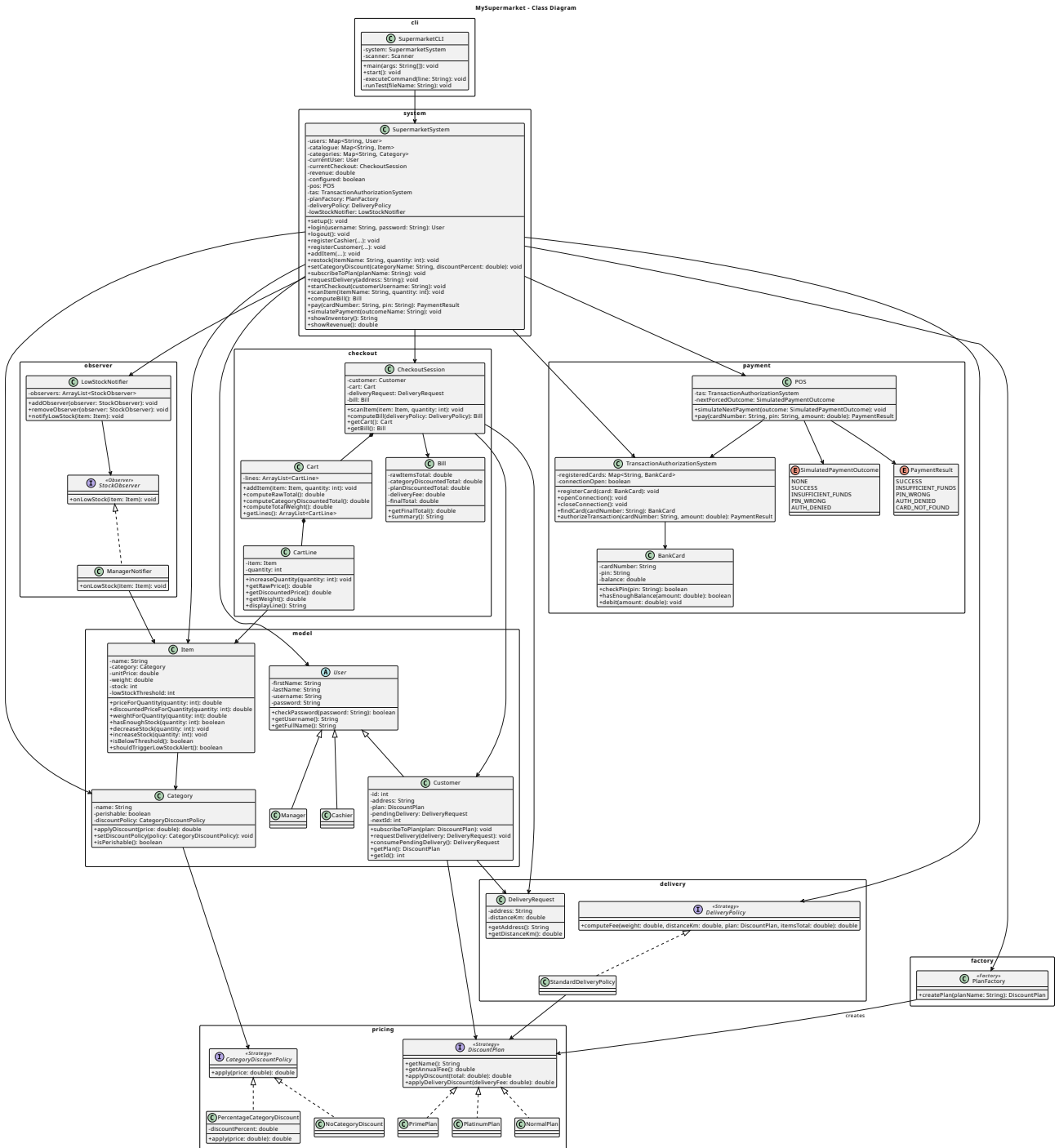


Figure 4.1: Class diagram of the Supermarket Checkout System.

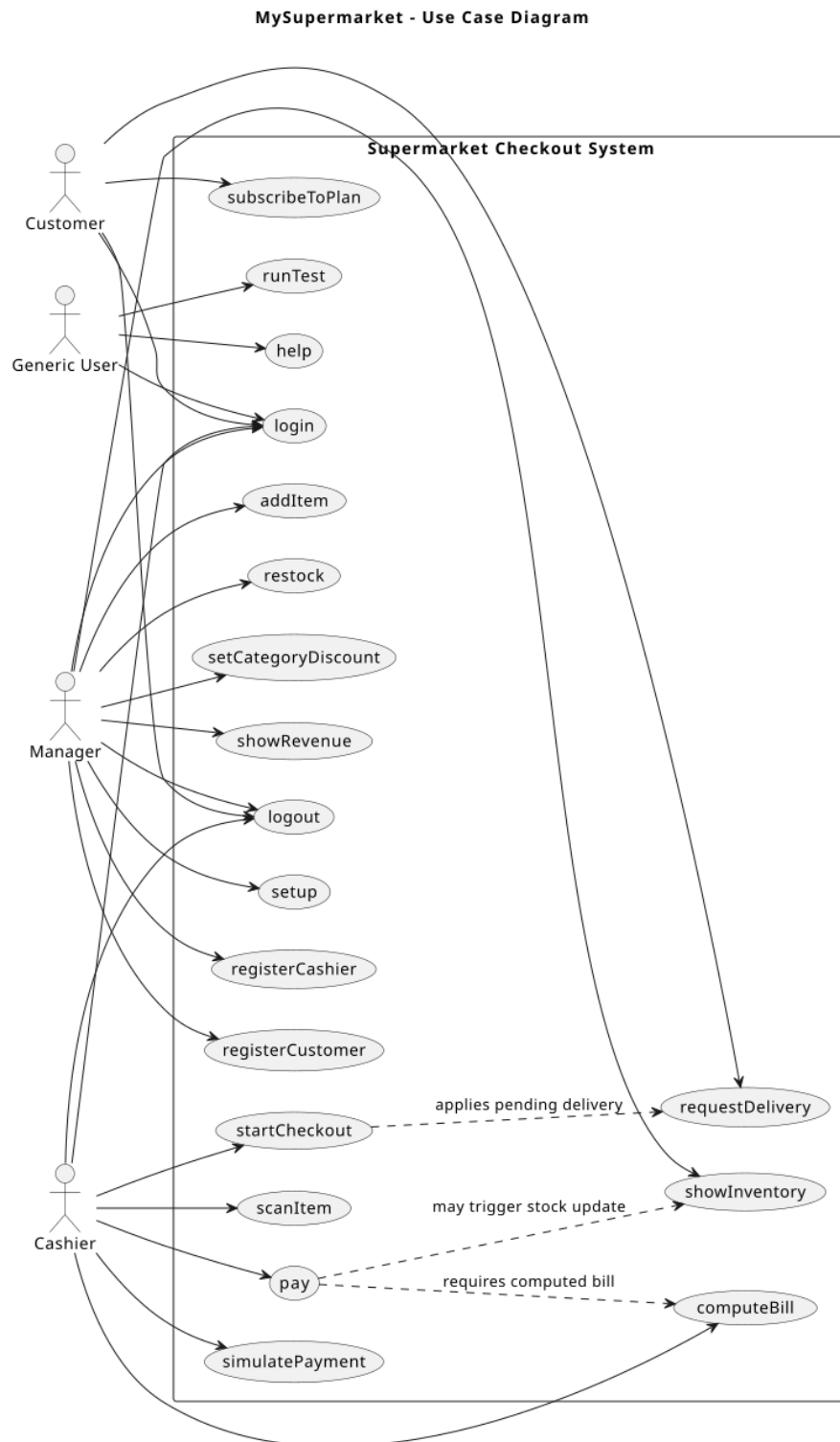


Figure 4.2: Use case diagram showing the system actors and their available CLUI commands.

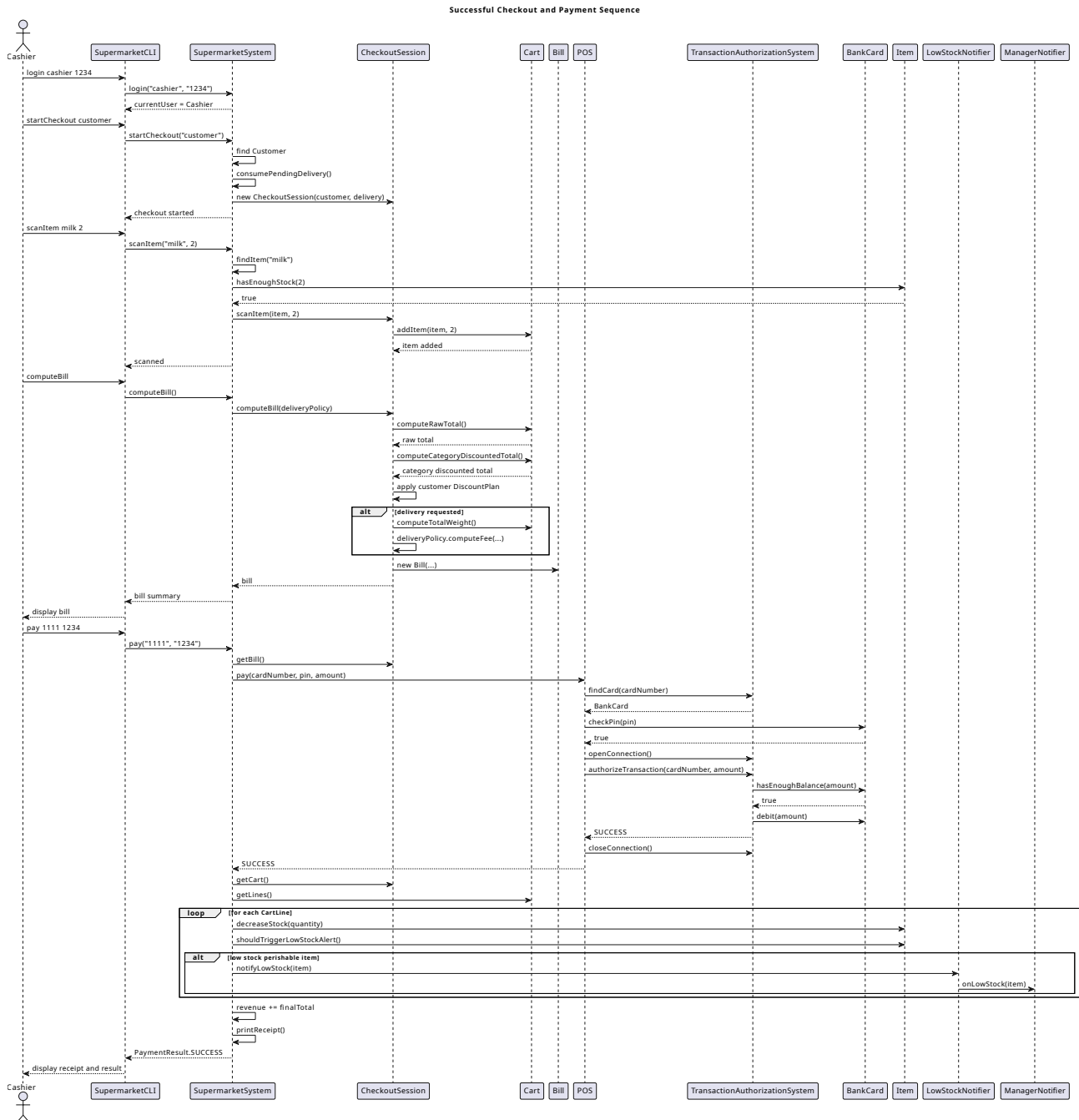


Figure 4.3: Successful checkout sequence diagram showing item scanning, total calculation, payment authorization, inventory update, and receipt printing.

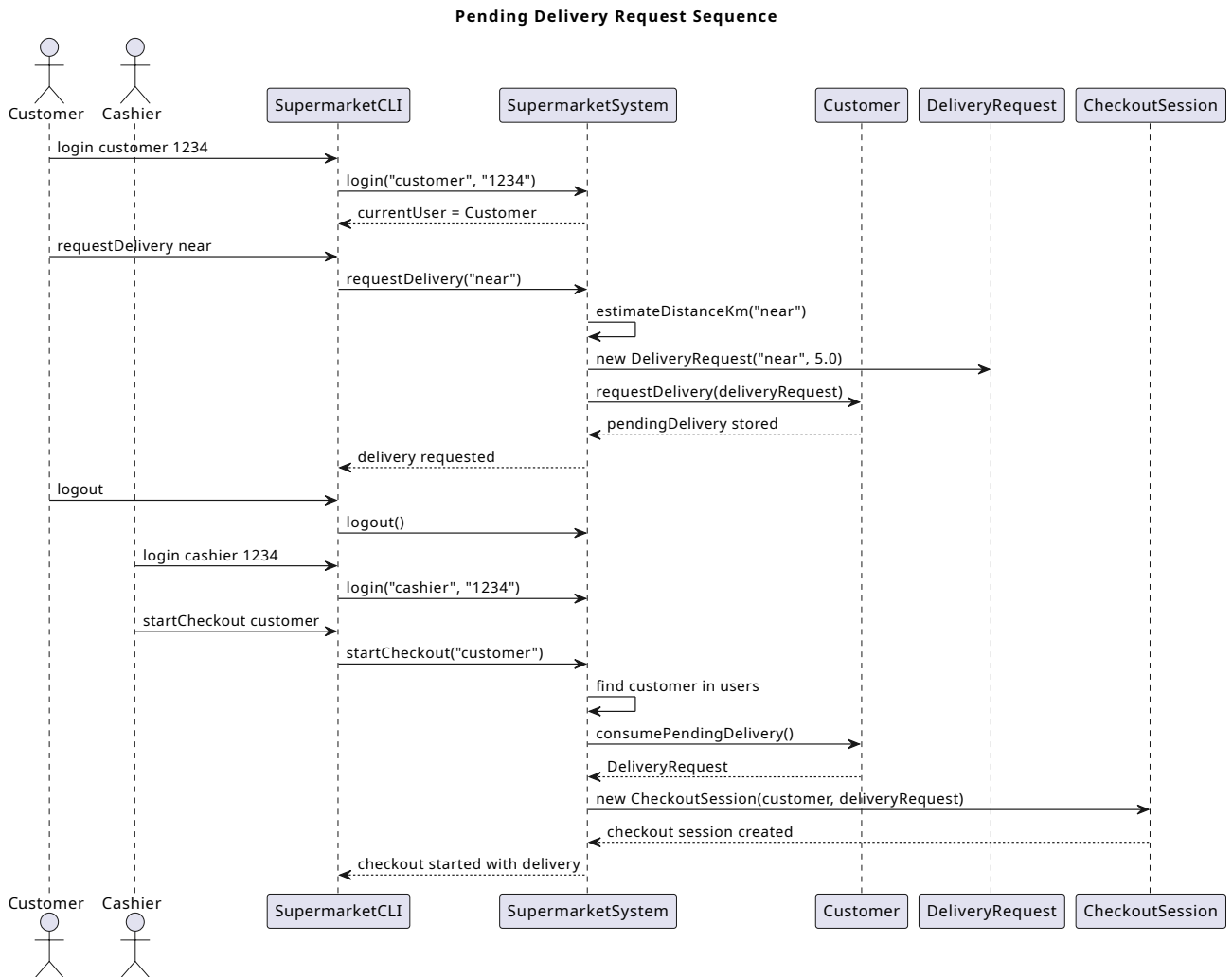


Figure 4.4: Delivery request sequence diagram showing how a customer creates a pending delivery request for the next purchase.

the provided address into a delivery distance. However, in this implementation, the address is simplified into predefined delivery zones: **near**, **medium**, and **far**.

The system then creates a **DeliveryRequest** and stores it in the customer object as a pending delivery. When a cashier later initiates the next checkout for that customer, the **SupermarketSystem** consumes the pending delivery request and passes it to the new **CheckoutSession**. This ensures that the delivery request is applied only to the next purchase.

4.5 Payment Failure Sequence Diagram

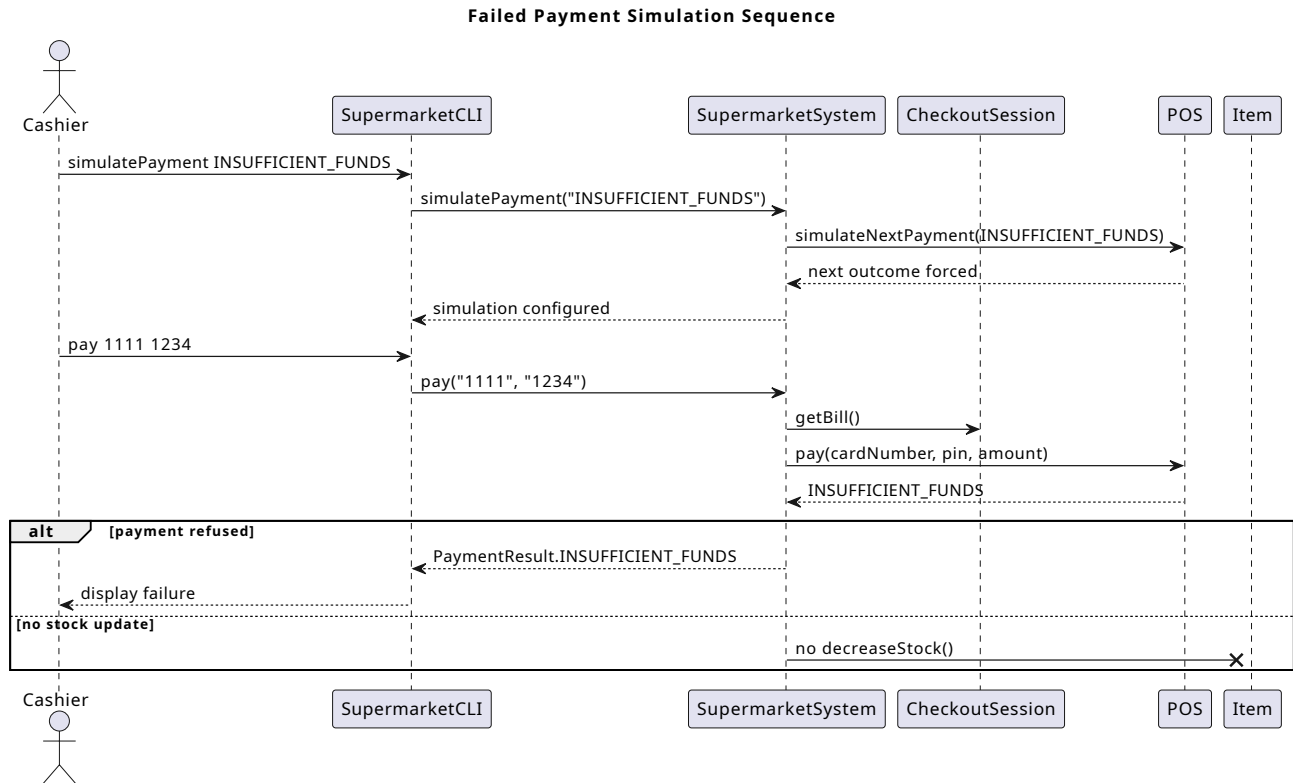


Figure 4.5: Payment failure sequence diagram showing simulated payment outcomes and the system behavior after a failed transaction.

The payment failure sequence diagram illustrates the use of the `simulatePayment` command, which allows the cashier to force the outcome of the next payment attempt. This action is useful for reproducing failure scenarios without manually modifying the bank card's state.

For example, the cashier can force the next payment to return `INSUFFICIENT_FUNDS`, `PIN_WRONG`, or `AUTH_DENIED`. When the payment fails, the system does not decrease inventory, does not increase revenue, and does not complete the checkout as a successful sale. This behavior is crucial, given that inventory should only be updated after a completed and successful sale.

Chapter 5

Testing and Test Scenario

5.1 Test scenario description

The project includes a CLUI test scenario file named `testScenario1.txt`. This file contains a sequence of command-line instructions that simulate a complete checkout scenario at a supermarket. The purpose of this file is to test the system's key features in a reproducible manner.

The scenario begins by loading the supermarket's default configuration, which includes the required item categories, default users, catalogue items, and pre-registered bank cards. The scenario then tests the interaction between two main user roles: the customer and the cashier. First, the customer logs in and requests delivery for the next purchase. Then, the cashier logs in, starts a checkout session for that customer, scans items, computes the bill, performs a successful payment, and finally checks the updated inventory and revenue.

The scenario covers the following behaviours:

- loading the default supermarket setup;
- logging in with different user roles;
- requesting delivery for the next purchase;
- starting a checkout session for a registered customer;
- scanning items into the current cart;
- computing the bill with delivery fees;
- performing a successful payment through the POS/TAS payment module;
- decrementing the inventory after a successful sale;
- displaying inventory and revenue.

Other behaviours, such as category-level discounts, customer plan subscription, payment failures with `simulatePayment`, and low-stock notifications, are covered mainly by the JUnit test suite. Therefore, the CLUI scenario is used as an integration test for the main successful checkout flow, while the JUnit tests are used to verify more specific behaviours independently.

The `runTest` command ignores empty lines and lines beginning with `#`. This allows the scenario file to contain comments explaining each step.

The scenario can be executed with the following command:

```
runTest testScenario1.txt
```

This makes it easier to reproduce the test scenario, since the tester only needs to open the CLUI and run the command above.

5.2 How to run the CLUI scenario

To run the application manually, the user must execute the `SupermarketCLI` class as a Java application.

In Eclipse, the steps are:

1. Open the project.

2. Open `cli/SupermarketCLI.java`.
3. Right-click the file.
4. Select `Run As → Java Application`.

After the CLUI starts, the user can type commands directly in the console. For example:

```
login ceo 123456789
setup
setCategoryDiscount dairy 10
logout
login cashier 1234
startCheckout customer
scanItem milk 2
computeBill
pay 1111 1234
```

To execute the predefined test scenario automatically, the user can type:

```
runTest testScenario1.txt
```

The file `my_supermarket.ini` is used as the initial configuration file. It ensures that, at startup, the system contains at least the default setup required by the project statement. The file `testScenario1.txt` is then used to reproduce a complete scenario through the CLUI.

The CLUI also displays error messages when commands are used incorrectly. For example, trying to scan an item before opening a checkout session, paying with an invalid card, or using a manager-only command as a customer will produce an error message instead of stopping the program.

5.3 JUnit tests

In addition to the CLUI scenario, the project includes JUnit tests for the implementation's most important methods. Trivial getter and setter methods were not the primary focus of the tests, as they would not be significant in evaluating the system. The purpose of the JUnit tests is to verify the behavior of each important class independently, while the CLUI scenario verifies the system's complete integration.

The main tested parts are summarized in Table 5.1.

Package	Main tested classes	Tested behaviour
pricing	NormalPlan, PrimePlan, PlatinumPlan, PercentageCategoryDiscount	Customer plan discounts and category discounts
factory	PlanFactory	Creation of the correct plan from a plan name
model	Item, Customer, Category	Stock updates, pending delivery, category policy, low-stock condition
checkout	Cart, CartLine, CheckoutSession, Bill	Item scanning, cart totals, bill computation
delivery	DeliveryRequest, StandardDeliveryPolicy	Delivery fee computation and refusal of unsupported deliveries
payment	BankCard, POS, TransactionAuthorizationSystem	PIN checking, balance checking, authorization, failed payments
observer	LowStockNotifier, ManagerNotifier	Low-stock notification mechanism
system	SupermarketSystem	End-to-end workflow, permissions, checkout, payment and revenue

Table 5.1: Summary of the main JUnit tests.

To run the JUnit tests in Eclipse:

1. Open the test package.
2. Right-click the test folder or a specific test class.
3. Select **Run As → JUnit Test**.

The expected result is that all JUnit tests pass successfully. If a test fails, Eclipse shows the failed assertion and the corresponding test method, making it easier to identify the class or behaviour that needs correction.

The testing strategy combines two complementary approaches.

First, JUnit tests check individual classes and important methods in isolation. This helps verify specific behaviours such as discount computation, stock decrement, delivery fee calculation, payment authorization, and observer notification.

Second, the `testScenario1.txt` file checks the complete system through the CLUI. This validates that the different modules work together correctly: user login, manager configuration, customer delivery request, cashier checkout, bill computation, payment, inventory update, revenue update, and notification. This makes the implementation easy to test and reproduce. A tester can either run the JUnit test suite to verify the internal logic or run:

```
runTest testScenario1.txt
```

to reproduce a complete user scenario from the command line.

Chapter 6

Workload Distribution

The workload was divided between Amanda and João Vitor according to the main packages and tasks of the project. Amanda contributed more strongly to the design patterns, UML diagrams, pricing and delivery modules, and a larger part of the JUnit tests. João Vitor contributed more strongly to the model package, observer logic, scenario files, and part of the system integration.

Class / Task	Design	Code	UML diagrams	JUnit test	Comments
checkout package	Amanda + João Vitor	Amanda + João Vitor	Amanda	Amanda + João Vitor	Cart, CartLine, Bill, and CheckoutSession.
cli package	João Vitor	Amanda + João Vitor	Amanda	João Vitor	Command parsing, help command and <code>runTest</code> .
delivery package	Amanda	Amanda	Amanda	Amanda	Delivery request, delivery zones and delivery fee policy.
factory package	Amanda	Amanda	Amanda	Amanda	Factory used to create discount plans from the CLUI input.
model package	João Vitor	João Vitor	Amanda	João Vitor	Users, customers, categories, items, stock and pending delivery.
observer package	João Vitor + Amanda	João Vitor	Amanda	Amanda	Observer mechanism for low-stock notifications.
payment package	Amanda + João Vitor	Amanda + João Vitor	Amanda	Amanda + João Vitor	Bank cards, POS, TAS and payment results.
pricing package	Amanda	Amanda	Amanda	Amanda	Customer plans and category discount policies.
system package	Amanda + João Vitor	Amanda + João Vitor	Amanda + João Vitor	Amanda + João Vitor	Central coordination of users, checkout, payment, delivery, and inventory.
<code>my_supermarket.ini</code> and <code>testScenario1.txt</code>	João Vitor	João Vitor	–	João Vitor	Initial configuration file and reproducible CLUI scenario.
JUnit test files	Amanda + João Vitor	Amanda + João Vitor	–	Amanda	Amanda wrote most tests; João Vitor contributed integration and scenario tests.
UML diagrams	Amanda + João Vitor	–	Amanda	–	Amanda produced the final diagrams, with validation from João Vitor.
Report writing	Amanda + João Vitor	–	Amanda	Amanda + João Vitor	Amanda contributed structure, diagrams, and formatting; João Vitor contributed technical and testing explanations.

Table 6.1: Workload distribution between Amanda and João Vitor.